THREE SIGMA

[un]wind

Security Review

Disclaimer

Security Review

[un]wind

Disclaimer

The ensuing audit offers no assertions or assurances about the code's security. It cannot be deemed an adequate judgment of the contract's correctness on its own. The authors of this audit present it solely as an informational exercise, reporting the thorough research involved in the secure development of the intended contracts, and make no material claims or guarantees regarding the contract's post-deployment operation. The authors of this report disclaim all liability for all kinds of potential consequences of the contract's deployment or use. Due to the possibility of human error occurring during the code's manual review process, we advise the client team to commission several independent audits in addition to a public bug bounty program.

Table of Contents

Security Review

[un]wind

Table of Contents

Summary

Security Review

[un]wind

Summary

Three Sigma audited Keyring in a 2.4 person week engagement. The audit was conducted from 4/06/2026 to 11/06/2026.

Protocol Description

RWA Unwind is a protocol that manages the full lifecycle of leveraged positions on tokenized Real-World Assets (RWAs) within Euler V2 lending markets. The system consists of two core modules, Wind and Unwind, that enable users to atomically enter and exit leveraged RWA positions despite the asynchronous settlement nature of the underlying assets. During a Wind, the protocol deploys per-request escrow contracts that mint a temporary interim collateral token (PSBA), borrow from Euler, and queue deposits into Pareto credit vaults (Idle Finance); once the RWA shares are claimable, an operator settles the queue and finalizes each position by swapping the interim collateral for real RWA shares and transferring the CDP to the user. During an Unwind, the reverse process locks the user's CDP, replaces RWA collateral with PSBA, queues a redemption through Pareto, and upon settlement returns the underlying proceeds minus fees. The protocol integrates with the Ethereum Vault Connector (EVC) for batched vault operations, Pyth Network for price feed validation, and Keyring Network for KYC gating, and relies on role-gated operators to coordinate queue settlement and liquidation of unhealthy positions.

Scope

Security Review

[un]wind

Scope

Filepath	nSLOC
contracts/src/wind/euler/core/WindManager.sol	598
contracts/src/unwind/euler/core/UnwindManagerExtension.sol	445
contracts/src/unwind/euler/core/UnwindManager.sol	366
contracts/src/unwind/euler/core/UnwindLiquidationAuction.sol	242
contracts/src/wind/euler/core/WindEscrow.sol	197
contracts/src/unwind/euler/core/UnwindEscrow.sol	195
contracts/src/unwind/euler/core/UnwindManagerFactory.sol	186
contracts/src/wind/euler/adapters/AssetoAdapter.sol	180
contracts/src/unwind/euler/adapters/AssetoAdapter.sol	177
contracts/src/wind/euler/core/WindManagerFactory.sol	123
contracts/src/unwind/euler/core/UnwindLiquidator.sol	93
contracts/src/common/UserWallet.sol	78
contracts/src/common/UserWalletFactory.sol	78
contracts/src/unwind/euler/core/UnwindManagerStorage.sol	76
contracts/src/common/FeeCollector.sol	32
Total	**3066**

Methodology

Security Review

[un]wind

To begin, we reasoned meticulously about the contract's business logic, checking security-critical features to ensure that there were no gaps in the business logic and/or inconsistencies between the aforementioned logic and the implementation. Second, we thoroughly examined the code for known security flaws and attack vectors. Finally, we discussed the most catastrophic situations with the team and reasoned backwards to ensure they are not reachable in any unintentional form.

Taxonomy

In this audit, we classify findings based on Immunefi's Vulnerability Severity Classification System (v2.3) as a guideline. The final classification considers both the potential impact of an issue, as defined in the referenced system, and its likelihood of being exploited. The following table summarizes the general expected classification according to impact and likelihood; however, each issue will be evaluated on a case-by-case basis and may not strictly follow it.

Impact / Likelihood	LOW	MEDIUM	HIGH
NONE	None		
LOW	Low		
MEDIUM	Low	Medium	Medium
HIGH	Medium	High	High
CRITICAL	High	Critical	Critical

Project Dashboard

Security Review

[un]wind

Project Dashboard

Application Summary

Name	Keyring
Repository	https://github.com/Keyring-Network/rwa-unwind
Commit	8c35b5d8e89202bcf796006a829af4cd454594c6
Language	Solidity
Platform	Avalanche

Engagement Summary

Timeline	4/06/2026 to 11/06/2026
№ of Auditors	2
Review Time	2.4 person weeks

Vulnerability Summary

Issue Classification	Found	Addressed	Acknowledged
Critical	0	0	0
High	1	1	0
Medium	5	4	1
Low	2	2	0
None	2	2	0

Category Breakdown

Suggestion	2
Documentation	0
Bug	0
Optimization	0
Good Code Practices	0

Risk Section

Security Review

[un]wind

Risk Section

- **Issue 3S-H01 Residual risk**: The fix mitigates this issue through operational coordination rather than an on-chain guarantee, so the underlying constraint is managed but not eliminated. Because the shared rwaCollateralVault accrues one SAmMMF queue entry per distinct depositor and the token caps each holder at MAX_QUEUE_LENGTH = 50 lots, the vault retains a finite ceiling on concurrent distinct users, and the accepted approach raises and monitors that ceiling instead of removing it. Capacity relief depends on Asseto's forthcoming upgrade to make MAX_QUEUE_LENGTH configurable and readable, and on Keyring's off-chain monitoring, which is reactive: there is an inherent race between detecting saturation and an on-chain capacity increase taking effect, and since settlement and finalization are separate transactions an available slot can be consumed in between. This residual is accepted on the basis that no more than roughly ten concurrent users are expected, leaving a large safety margin against the cap, and that adversarial acceleration is bounded by minting of new-provenance lots being gated behind Asseto's admin-mediated subscription, even though transferring an already-minted lot directly into the vault is permissionless.

Findings

Security Review

[un]wind

Findings

3S-H01

Aggregating per-user SAmMMF lots in one shared collateral vault hits the token's 50-entry queue cap and blocks wind finalization for new users

Id	3S--H01
Classification	High
Impact	High
Likelihood	High
Category	Bug
Status	Mitigated

Description

WindEscrow::finalizeWind finalizes a wind position by depositing its RWA collateral, the Asseto **SAmMMF** token, into the shared Euler **rwaCollateralVault** (**IEVault.deposit** at **src/wind/euler/core/WindEscrow.sol:170**). A single **rwaCollateralVault** backs every wind position, so all users' **SAmMMF** is aggregated into one address. The live **SAmMMF** does not track a single fungible balance per address. It keeps a per-wallet FIFO queue of lots, each tagged with a provenance **tokenId** equal to **keccak256(minter, chainid)**, and **SAmMMF::_depositInternal** caps a wallet at **MAX_QUEUE_LENGTH = 50** distinct lots, reverting **Wallet queue full** on the 51st (deposits of an already-present **tokenId** merge instead of consuming a slot). Because **SAmMMF** is minted to a per-user **_userWallet**, each user's lot carries a distinct **tokenId** that is preserved when the collateral is moved into the vault, so the vault accrues one queue entry per distinct user. Once 50 distinct users hold collateral, the 51st user's **finalizeWind** deposit reverts.

The condition has no protocol-controlled recovery while the queue stays full. **WindManager::settleRequest** has by then moved the user's claimed **SAmMMF** into the manager and recorded a non-zero **s_requestIdToClaimableShares** (**src/wind/euler/core/WindManager.sol:735**); the only consumer of those shares is **WindManager::_finalizeWind**, which is the reverting call, and **WindManager::safetyCloseWind** reverts while claimable shares are non-zero (**WindManager.sol:486-488**). With no sweep over **s_requestIdToClaimableShares**, the

already-claimed **SAmMMF** is stranded in the manager unless a later unrelated withdrawal frees a slot.

Recommendation

Introduce a wrapper token that becomes the underlying asset of **rwaCollateralVault** in place of raw **SAmMMF**. The wrapper is a conventional flat-balance ERC20, so a holder's balance is a single accumulator with no lot queue, no per-wallet entry cap, and no FIFO consumption order. Because **WindEscrow::finalizeWind** already reads the vault's underlying through **s_rwaCollateralVault.asset()** before depositing, swapping the underlying to the wrapper keeps the existing deposit and withdraw call shape intact and confines the change to the wrap and unwrap boundary.

Back the wrapper with one dedicated **SAmMMF** holder per user or per position rather than a single shared custodian. Each holder receives only its own user's lots, which all carry the same **tokenId**, so no holder ever holds more than approximately one queue entry and none of them can approach the 50-entry cap. At **finalizeWind**, the position's **SAmMMF** is routed into that position's holder and the wrapper mints an equal amount of flat shares that are deposited into the vault. At unwind, **UnwindEscrow::prepareRedemption** withdraws wrapper shares, the wrapper burns them, and the matching holder releases exactly that user's **SAmMMF** for redemption through Asseto.

3S-M01

Unconditional EVC operator cleanup during unwind finalization allows a receipt owner to block his liquidation

Id	3S--M01
Classification	Medium
Impact	Medium
Likelihood	Medium
Category	Bug
Status	Addressed in #766f7d6.

Description

When an unwind request is liquidated through the auction, the winning bidder does not receive funds immediately. Their repay assets are consumed up front to reduce the escrow's debt, and their payout is recorded as a deferred claim that can only be released once the manager reports the request as **Finalized**. **UnwindLiquidationAuction::claimWinningRoundPayout** enforces this ordering: it refuses to pay the winner until finalization has occurred. Reaching **Finalized** therefore depends on **UnwindManagerExtension::finalizeUnwind** completing successfully, which in turn calls into the escrow's **UnwindEscrow::_withdrawCdp** to hand any residual position back to the requester's sub-account and clear the escrow's operator authorization on that sub-account.

The problem is that **UnwindEscrow::_withdrawCdp** treats the residual-debt transfer and the operator cleanup asymmetrically. The debt **pullDebt** batch item is only appended when there is leftover debt to move, but the **setAccountOperator** call that deauthorizes the escrow on the sub-account is appended unconditionally. Because batch items that target the EVC are executed such that the escrow remains the authenticated caller, **setAccountOperator** only succeeds when the escrow is the account owner or is currently an authorized operator acting on itself. The sub-account is controlled by the requester, who remains the receipt NFT owner even after their position has been liquidated, since liquidation does not seize the receipt. That owner can revoke the escrow's operator authorization at any point, or simply never grant it, before finalization runs. When the operator bit is not set, the unconditional deauthorization item reverts the entire finalization batch, even though there is no residual debt to move and the operator entry is merely being cleaned up as a no-op.

The impact is that a liquidated borrower can permanently block the settlement of their own liquidation and deny the auction winner a payout for capital the winner has already committed.

Recommendation

Make **UnwindEscrow::_withdrawCdp** append the operator-cleanup batch item only when the escrow is currently an authorized operator on the recipient sub-account, mirroring the existing conditional treatment of the residual-debt transfer. Query the EVC for the escrow's operator authorization on the sub-account and skip the **setAccountOperator** deauthorization entirely when it is not set, so that finalization no longer depends on a state the requester controls.

3S-M02

UnwindLiquidationAuction::submitBid lacks a minimum bonus decrement step, allowing 1-wei bid sniping

Id	3S--M02
Classification	Medium
Impact	Medium
Likelihood	High
Category	Bug
Status	Addressed in #386ad99.

Description

UnwindLiquidationAuction::submitBid accepts a reverse-bonus bid for an open liquidation round, where a lower **_bonus** wins. Each new bid must improve on the current best bid, and the contract enforces this with a single check:

```
if (round.bestBidder != address(0) && _bonus >= round.bestBonus) {
    revert UnwindLiquidationAuction__BidNotImproved(_bonus, round.bestBonus);
}
```

The check only requires the new **_bonus** to be strictly less than **round.bestBonus**. It places no lower bound on the decrement, so any bidder can outbid the current best bidder by lowering **round.bestBonus** by a single wei. The bonus can move by arbitrarily small amounts, so competition never drives it down toward an efficient market price.

The missing decrement step compounds the sniping problem around **round.auctionEnd**. **submitBid** reverts once **block.timestamp >= round.auctionEnd**. An attacker watching the mempool waits for the final moments of the round, then submits a bid at **round.bestBonus - 1**. The attacker wins for the smallest possible improvement, and honest bidders who bid earlier have no time to respond. Honest bidders stop competing. The protocol settles the liquidation at a bonus only marginally better than the first bid instead of a genuinely competitive price.

Recommendation

Require each new bid to improve on the current best bonus by a minimum percentage. Store the percentage in basis points (**s_minBonusDecrementBps**), mirroring the existing **s_bidBondBps**, and compute the absolute minimum decrement from **round.bestBonus** when **submitBid** runs. This scales the required step to the size of the current bonus rather than fixing it to an absolute amount that becomes too large or too small as **round.bestBonus** moves.

```
+       uint256 minDecrement = (round.bestBonus * s_minBonusDecrementBps) / BPS;
-       if (round.bestBidder != address(0) && _bonus >= round.bestBonus) {
+       if (round.bestBidder != address(0) && _bonus + minDecrement > round.bestBonus) {
            revert UnwindLiquidationAuction__BidNotImproved(_bonus, round.bestBonus);
        }
```

Add the backing state and validate it on construction the same way as **s_bidBondBps**.

```
    uint16 internal s_bidBondBps;
+   uint16 internal s_minBonusDecrementBps;
```

```
        if (_params.bidBondBps > BPS) revert
UnwindLiquidationAuction__InvalidBidBondBps(_params.bidBondBps);
+       if (_params.minBonusDecrementBps > BPS) {
+           revert
UnwindLiquidationAuction__InvalidMinBonusDecrementBps(_params.minBonusDecrement
Bps);
+       }
        ...
        s_bidBondBps = _params.bidBondBps;
+       s_minBonusDecrementBps = _params.minBonusDecrementBps;
```

3S-M03

EVC-resolved **_msgSender()** in inherited ERC721 authorization lets an EVC operator transfer a user's receipt NFT and capture the unwind proceeds

Id	3S--M03
Classification	Medium
Impact	High
Likelihood	Low
Category	Bug
Status	Addressed in #79a8f56.

Description

UnwindManager mints a receipt NFT for every unwind request and inherits OpenZeppelin's **ERC721EnumerableUpgradeable** to manage ownership of those NFTs. The receipt NFT is the bearer of the request's economic value: at finalization, **UnwindManagerExtension::finalizeUnwind** pays the surplus left after debt repayment and Keyring fees to the current NFT holder (**nftOwner**). Like the rest of the contract, **UnwindManager** overrides **_msgSender()** to resolve the caller through the EVC (**return EVCUtilUpgradeable._msgSender();**), so any function that authorizes via **_msgSender()** trusts the account on whose behalf the EVC is operating rather than the direct **msg.sender**.

The ERC721 mutating entrypoints (**transferFrom**, **safeTransferFrom**, **approve**, and **setApprovalForAll**) are inherited without override and resolve their authorizing principal through **_msgSender()**. OpenZeppelin's **ERC721Upgradeable::transferFrom** passes **_msgSender()** as the **auth** argument to **_update**, which checks **_isAuthorized(owner, spender, tokenId)**, and that predicate returns true when **owner == spender**. Because the override makes **_msgSender()** return the EVC on-behalf-of account, an EVC operator authorized for the NFT owner can call **transferFrom** (or **safeTransferFrom**) through the EVC; **_msgSender()** resolves to the owner, the **owner == spender** branch is satisfied, and the transfer succeeds even though the operator holds no ERC721 token approval and no **setApprovalForAll** grant. The operator can move the receipt NFT to an address it controls and then drive the request to finalization, becoming the **nftOwner** that collects the surplus assets.

EVC operators are routinely authorized for narrower duties such as keeper bots or sub-account automation, with no expectation that the grant also confers the ability to move a user's receipt NFT. An operator exceeding that scope can take the NFT without the

owner's explicit ERC721 approval and unwind the position to its own benefit, draining the user's recoverable assets. The wind module's receipt NFT (**WindManager**) inherits the same EVC-aware pattern and is exposed to the equivalent transfer-then-finalize sequence.

Recommendation

Authorize the receipt NFT's transfer and approval surface against the direct **msg.sender** rather than the EVC-resolved **_msgSender()**, mirroring **UnwindManager::acceptTos** and the **_checkRoleMsgSender** fix from issue #48. Override **transferFrom**, **safeTransferFrom**, **approve**, and **setApprovalForAll** so the authorizing principal is **msg.sender**, which preserves normal owner and ERC721-approved-operator flows while preventing EVC operators from acting on the token solely by virtue of their operator status.

3S-M04

Auction liquidation continues against a settled unwind request in **UnwindManagerExtension::checkAuctionLiquidation** and **UnwindManagerExtension::executeAuctionLiquidation**

Id	3S--M04
Classification	Medium
Impact	Medium
Likelihood	Medium
Category	Bug
Status	Addressed in .

Description

UnwindManagerExtension::checkAuctionLiquidation and **UnwindManagerExtension::executeAuctionLiquidation** gate the liquidation auction. The former reports whether an unwind request is liquidatable. The latter funds the seizure through the shared liquidator. **UnwindLiquidationAuction::openLiquidationRound** calls **UnwindManagerExtension::checkAuctionLiquidation**, and **UnwindLiquidationAuction::executeWinningRound** calls **UnwindManagerExtension::executeAuctionLiquidation**.

Both functions accept the request as long as the escrow status equals **IUnwindEscrow.Status.Redeeming**. The escrow status only advances to **Finalized** inside **UnwindEscrow::finalizeUnwind**. When **UnwindManager::_settleRequestIfClaimable** claims the redemption through the adapter, it credits **s_requestIdToClaimableAssets[_requestId]** with the redeemed assets but leaves the escrow status at **Redeeming**. So a request that has already been settled still passes the **status != IUnwindEscrow.Status.Redeeming** guard in both functions.

Once settlement claims the redemption, the PSBA backing the Euler CDP no longer maps to the RWA. The auctioned asset no longer exists, so the CDP is closed and the liquidation must stop. Neither function reads **s_requestIdToClaimableAssets**, so the health check in **UnwindManagerExtension::checkAuctionLiquidation** runs against the stale PSBA collateral value inside the CDP rather than the redeemed USDC the request now holds. When that stale PSBA valuation reports the position as underwater, **UnwindLiquidationAuction::executeWinningRound** succeeds against a position that is healthy in redeemed-asset terms, and the borrower loses the bonus paid to the auction winner. The auction stays open even when the redeemed value sits below the liquidation threshold, which is wrong because the auctioned RWA no longer exists.

The auction side carries a second consequence. The best bidder can wait for the request to settle and then execute the round to collect the bonus with no risk of bond slashing, because **UnwindLiquidationAuction::expireRound** only slashes rounds that pass the deadline without execution.

Steps to reproduce

1. A borrower opens an unwind request and the escrow enters **Redeeming**.

2. A keeper calls **UnwindManager::settleRequest**, which claims the redemption through the adapter and credits **s_requestIdToClaimableAssets[_requestId]** with the redeemed assets while the escrow status stays **Redeeming**.

3. The stale PSBA valuation inside the Euler CDP reports the position as underwater, even though the redeemed USDC now backing the request exceeds the liquidation threshold.

4. The auction manager calls **UnwindLiquidationAuction::openLiquidationRound**. **UnwindManagerExtension::checkAuctionLiquidation** still sees **status == IUnwindEscrow.Status.Redeeming** and reports the request as liquidatable.

5. Bidders submit reverse-bonus bids and the auction proceeds.

6. After the auction ends, the best bidder calls **UnwindLiquidationAuction::executeWinningRound**, which calls **UnwindManagerExtension::executeAuctionLiquidation**. The settled-status check is absent, so the seizure executes and the borrower loses the bonus on a position that is healthy in redeemed-asset terms.

Recommendation

Reject auction checks and execution once a request has been settled by treating any nonzero **s_requestIdToClaimableAssets[_requestId]** as a closed position. Add a new error to **IUnwindManagerExtension** and apply the guard to both functions.

Note: this guard only detects settlement that has been recorded in the adapter through **s_requestIdToClaimableAssets**. A request whose redemption is ready on Asseto but not yet claimed here is not covered, because the Asseto adapter exposes no reliable redemption-ready view. Closing that residual gap requires a reliable settlement signal from Asseto and is out of scope of this fix.

contracts/src/unwind/euler/core/interfaces/IUnwindManagerExtension.sol

```
+    /// @notice Thrown when an auction action targets an already-settled request.
+    /// @param _unwindRequestId The unwind request identifier.
+    error UnwindManagerExtension__UnwindRequestAlreadySettled(bytes32
```

_unwindRequestId);

contracts/src/unwind/euler/core/UnwindManagerExtension.sol

```
        IUnwindEscrow.Status status = unwindEscrow.getEscrowState().status;
        if (status != IUnwindEscrow.Status.Redeeming) return (false, 0, status);
+        if (s_requestIdToClaimableAssets[_requestId] != 0) return (false, 0, status);



        IUnwindEscrow.Status status = unwindEscrow.getEscrowState().status;
        if (status != IUnwindEscrow.Status.Redeeming) {
            revert UnwindManagerExtension__UnwindRequestNotRedeeming(status);
        }
+        if (s_requestIdToClaimableAssets[_requestId] != 0) {
+            revert UnwindManagerExtension__UnwindRequestAlreadySettled(_requestId);
+        }
```

3S-M05

Mutually exclusive exit conditions after settlement lead to permanent freezing of user collateral and lender funds

Id	3S--M05
Classification	Medium
Impact	High
Likelihood	Low
Category	Bug
Status	Acknowledged

Description

UnwindManager::_settleRequestIfClaimable settles an unwind request by claiming the redeemed stablecoins from the RWA adapter and recording them in **s_requestIdToClaimableAssets**. The function accepts any non-zero **redeemedAssets** value; it only reverts when the claim returns zero, and the user has no **minAssets** or **minShares** parameter to bound the outcome. Once a positive amount is recorded, the early return on **s_requestIdToClaimableAssets[_requestId] != 0** permanently blocks re-settlement of the request.

After settlement, the request has exactly two exit paths, and their preconditions are mutually exclusive. **UnwindManagerExtension::finalizeUnwind** reverts with **UnwindManagerExtension__InsufficientAssetsToRepayDebt** whenever **claimableAssets** is below the escrow's live debt read from **IEVault::debtOf**. **UnwindManagerExtension::safetyCloseUnwind**, the SAFETY_MANAGER rescue path, reverts with **UnwindManagerExtension__SafetyCloseUnavailable** whenever **claimableAssets** is non-zero. Consequently, for any settled amount in the range **0 < claimableAssets < debt**, neither function can execute, and no other admin or operator function can top up the shortfall or clear the recorded state. The escrowed collateral, the minted pending settlement base asset, and the settled stablecoins are frozen indefinitely while debt interest continues to accrue, widening the gap and making recovery impossible.

This state is reachable without any malicious actor, because **requestUnwind** sizes the redemption to the collateral rather than the debt and the protocol never enforces **claimableAssets >= debt** until finalization, after the request is already irreversibly marked settled. Several mechanisms could drive the payout below the live debt, whether they apply today or only become relevant as the protocol's integrations evolve. A NAV decline or haircut on the Asseto SAmMMF money-market fund would deliver less USD for the same tokens if it marks down or applies an unfavorable price at fulfillment time. Redemption fees

or spread deducted by Asseto would be booked by the adapter as a smaller balance delta without complaint. A partial fulfillment by Asseto would be recorded by **_settleRequestIfClaimable** and then locked via the **claimableAssets != 0** early return, blocking the rest from ever being claimed. Borrow interest accruing during the asynchronous redemption window can trap even a full payout, since **debtOf** is read live at finalize time while **claimableAssets** was frozen at settlement, so a position unwound near its maximum LTV needs only the interest accrued between the settle block and the finalize block to fall into the trap.

The likelihood of any of these conditions occurring today is very low to non-existent, given the current parameters and the trusted, conservatively managed nature of the Asseto integration. Even so, the codebase should be hardened to handle the **0 < claimableAssets < debt** case gracefully rather than relying on those assumptions holding indefinitely, because the consequence is not a degraded outcome but a permanent freeze of user equity and lender liquidity.

Recommendation

Add an authorized recovery path for the settled-but-stuck state. Allow **UnwindManagerExtension::safetyCloseUnwind** (or a dedicated recovery function) to accept a positive but insufficient **claimableAssets** balance, apply it toward the escrow's debt, and require the safety manager to supply only the remaining shortfall before unwinding the pending settlement base asset and debt from the escrow's holdings.

3S-L01

EVC-resolved sender is allowed to renounce roles

Id	3S--L01
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Addressed in #86e3762.

Description

Both **WindManager** and **UnwindManager** derive from an EVC-aware base and globally replace the standard **ContextUpgradeable::_msgSender** with the EVC-aware **EVCUtilUpgradeable::_msgSender**, which resolves an EVC execution to the **onBehalfOfAccount** rather than the direct **msg.sender**. This means that inherited **AccessControl** logic sees the delegating account as the caller whenever a call is routed through the EVC by an authorized operator.

To defend against operators managing privileges on behalf of the accounts that delegated to them, both managers override **grantRole** and **revokeRole** so that the admin-role check is performed against the direct **msg.sender** instead of the EVC-resolved sender. However, the inherited **renounceRole** entrypoint receives no such treatment. OpenZeppelin's **renounceRole** is intentionally a self-service action: it only succeeds when the caller is the account whose role is being renounced. Because the managers feed it the EVC-resolved sender, a call routed through the EVC by an authorized operator appears to originate from the role holder itself, satisfying the self-confirmation requirement.

Recommendation

Override **renounceRole** in both **WindManager** and **UnwindManager** so that the self-confirmation check is performed against the direct **msg.sender** rather than the EVC-resolved sender, mirroring the hardening already applied to **grantRole** and **revokeRole**.

3S-L02

Missing upper bound on the opening liquidation bid's bonus allows a sole bidder to capture a request's entire post-debt surplus

Id	3S--L02
Classification	Low
Impact	Medium
Likelihood	Low
Category	Bug
Status	Addressed in #a52337e.

Description

UnwindLiquidationAuction::submitBid drives a reverse-bonus auction: bidders compete to liquidate an unhealthy unwind request by requesting the smallest debt-asset bonus, and the lowest **_bonus** wins. The only constraint the function places on **_bonus** is the strict-improvement check **_bonus >= round.bestBonus**, and that check is guarded by **round.bestBidder != address(0)**. The opening bid for a round therefore has no upper bound at all; the first bidder can pass any **_bonus**, and if no one outbids them the round settles at that value.

At execution, **UnwindLiquidationAuction::executeWinningRound** records **round.payoutAmount = actualRepay + round.bestBonus** and adds it to **s_unpaidReservedPayout[requestId]**, which the manager reads through **getUnpaidReservedPayout**. When the request is finalized in **UnwindManagerExtension::finalizeUnwind**, the surplus left after debt repayment and Keyring fees (**remainingAssets**) is split: an amount up to **unpaidLiquidationPayout** is set aside as the liquidation payout pool, and only the residual **remainingAssets - liquidationPayoutPool** is sent to the request owner (**nftOwner**). The winner later draws from that pool in **UnwindLiquidationAuction::claimWinningRoundPayout**, which calls **UnwindManagerExtension::releaseAuctionLiquidationPayout**. Because an inflated **bestBonus** inflates **unpaidLiquidationPayout**, it directly enlarges the share of the surplus diverted into the payout pool and shrinks what the request owner receives, down to zero once the reserved payout meets or exceeds the available surplus.

A single uncontested bid is possible: during protocol bootstrap, when the auction policy gates participation to a small set of Keyring-credentialed accounts, when the debt asset is illiquid, or when the auction window is short, the first bidder may face no competition and can set an arbitrary self-serving bonus. The bid bond scales with **repayAssets** and is recovered by the winner, so it imposes no economic ceiling on the requested bonus.

Recommendation

Enforce an absolute cap on **_bonus** for every bid, including the opening one, expressed as a fraction of the round's **repayAssets**. Add a configurable **s_maxBonusBps** (analogous to **s_bidBondBps**) and reject bids whose bonus exceeds **round.repayAssets * s_maxBonusBps / BPS**, so the maximum liquidator profit is a bounded percentage of the amount repaid rather than the request's full surplus.

```
  function submitBid(uint256 _roundId, uint256 _bonus) external {
      _requireValidKeyringCredential(msg.sender);
      Round storage round = s_rounds[_roundId];
      _requireRoundOpen(_roundId, round);
      if (block.timestamp >= round.auctionEnd) revert
UnwindLiquidationAuction__AuctionEnded(_roundId);
+     uint256 maxBonus = (round.repayAssets * s_maxBonusBps) / BPS;
+     if (_bonus > maxBonus) revert
UnwindLiquidationAuction__BonusExceedsCap(_bonus, maxBonus);
      if (round.bestBidder != address(0) && _bonus >= round.bestBonus) {
          revert UnwindLiquidationAuction__BidNotImproved(_bonus, round.bestBonus);
      }
```

3S-N01

Redundant positive-repay guard in the auction liquidation path

Id	3S--N01
Classification	None
Category	Suggestion
Status	Addressed in #4350e41.

Description

UnwindManagerExtension::executeAuctionLiquidation computes **actualRepay** and then reverts with **UnwindManagerExtension__UnwindRequestNotLiquidatable** when **actualRepay == 0**. The subsequent transfer is wrapped in **if (actualRepay > 0)**, but any execution path that reaches this wrapper has already passed the **actualRepay == 0** revert, so the condition is always true. The branch can never evaluate false; it is dead code.

There is no security impact, but a guard that can never be false implies a reachable zero-repay state that does not exist, which costs reviewer attention and a small amount of gas. The transfer should run unconditionally.

Recommendation

Remove the redundant guard and perform the **SafeERC20.safeTransferFrom** unconditionally, since **actualRepay** is guaranteed non-zero at that point.

```
- if (actualRepay > 0) {
-     SafeERC20.safeTransferFrom(cache.debtAsset, msg.sender, s_sharedLiquidator, actualRepay);
- }
+ SafeERC20.safeTransferFrom(cache.debtAsset, msg.sender, s_sharedLiquidator, actualRepay);
```

3S-N02

UnwindLiquidationAuction::closeRoundIfNotLiquidatable and **UnwindLiquidationAuction::expireRound** gate harmless lifecycle transitions behind a privileged role

Id	3S--N02
Classification	None
Category	Suggestion
Status	Addressed in #aa70ad1.

Description

UnwindLiquidationAuction::closeRoundIfNotLiquidatable closes an open liquidation round once the underlying request is no longer liquidatable, leaving the posted bonds withdrawable. **UnwindLiquidationAuction::expireRound** retires an open round after its execution deadline passes and slashes the current winning bond to the manager fee collector. Both functions restrict the caller to **LIQUIDATION_AUCTION_MANAGER_ROLE** through **_checkRole**.

Neither function accepts an adversarial input that changes its outcome. **UnwindLiquidationAuction::closeRoundIfNotLiquidatable** advances the round to **Closed** only when **IUnwindAuctionManager::checkAuctionLiquidation** reports the request is not liquidatable, and reverts otherwise. The forwarded **_pythUpdateData** feeds the manager's own price check, so a caller cannot steer the result. **UnwindLiquidationAuction::expireRound** advances the round to **Expired** only after **block.timestamp** passes **round.executionDeadline**, and routes any slashed bond to **IUnwindAuctionManager::getFeeCollector**, a destination the caller does not control. On-chain state determines both transitions and the fund flow. The caller identity changes nothing.

Binding these transitions to **LIQUIDATION_AUCTION_MANAGER_ROLE** couples liveness to that role holder being online and willing to act. When the role holder is offline or unresponsive, a healthy position cannot return to **Closed** and its posted bonds stay locked. An expired round cannot be cleared, so its bidders keep waiting and **s_activeRoundId[round.requestId]** stays set, which blocks a new round from opening for that request. Making both functions permissionless lets any caller advance a round to its terminal state as soon as the on-chain conditions hold.

Recommendation

Remove the **_checkRole(LIQUIDATION_AUCTION_MANAGER_ROLE, msg.sender)** guard from both functions so any caller can advance a round once its conditions are met, since neither function exposes a caller-controlled path to harm.

```
function expireRound(uint256 _roundId) external {
-       _checkRole(LIQUIDATION_AUCTION_MANAGER_ROLE, msg.sender);
        Round storage round = s_rounds[_roundId];
        _requireRoundOpen(_roundId, round);
        if (block.timestamp <= round.executionDeadline) {
            revert UnwindLiquidationAuction__ExecutionDeadlineActive(_roundId);
        }
```

```
function closeRoundIfNotLiquidatable(uint256 _roundId, bytes[] calldata _pythUpdateData)
external payable {
-       _checkRole(LIQUIDATION_AUCTION_MANAGER_ROLE, msg.sender);
        Round storage round = s_rounds[_roundId];
        _requireRoundOpen(_roundId, round);
```